In [1]:
```
# install required packages
%pip install -q "transformers>=4.50,<5" accelerate safetensors
```

In [2]:
```
# Natural Language Processing Functionality
from transformers import AutoTokenizer

# Data Handling And Analysis
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt

# Utilities
import json
from pathlib import Path
```

In [3]:
```
# project folder
project_folder = Path("/content/drive/MyDrive/OCOM5300M_VLM_Benchmark")
roco_folder = project_folder

if not project_folder.is_dir():
    raise FileNotFoundError(f"Google Drive project folder not found: {project_folder}")
```

In [4]:
```
# training captions filename
training_captions_filename = roco_folder / "train_captions.csv"

# tokenizer output folder
tokenizer_folder = project_folder / "transformer" / "bert_tokenizer"

if not training_captions_filename.is_file():
    raise FileNotFoundError(f"Training captions not found: {training_captions_filename}")
```

In [5]:
```
def load_caption_data(filename):
    """Load captions and prepare the cleaned_caption column."""

    caption_data = pd.read_csv(filename, dtype={"ID": str})
```

```
    if "ID" not in caption_data.columns:
        raise ValueError(f"Missing ID column: {filename}")

    if "cleaned_caption" not in caption_data.columns:
        source_column = next(
            (name for name in ["caption", "Caption", "text", "Text"]
             if name in caption_data.columns),
            None,
        )
        if source_column is None:
            raise ValueError(f"Missing caption column: {filename}")
        caption_data["cleaned_caption"] = (
            caption_data[source_column].fillna("").astype(str)
            .str.lower()
            .str.replace(r"[^a-z0-9\s]", " ", regex=True)
            .str.replace(r"\s+", " ", regex=True)
            .str.strip()
        )

    caption_data["ID"] = caption_data["ID"].astype(str).str.strip()
    caption_data["cleaned_caption"] = caption_data["cleaned_caption"].fillna("").astype(str).str.strip()

    if caption_data["ID"].eq("").any() or caption_data["cleaned_caption"].eq("").any():
        raise ValueError(f"Empty image IDs or captions found in {filename}")

    return caption_data
```

In [6]:

```
# Load training captions
training_caption_data = load_caption_data(training_captions_filename)

print("Training captions:", len(training_caption_data))
display(training_caption_data[["ID", "cleaned_caption"]].head())
```

```
Training captions: 59958
```

	ID	cleaned_caption
0	ROCOv2_2023_train_000001	head ct demonstrating left parotiditis
1	ROCOv2_2023_train_000002	acquired renal cysts in end stage renal failur...
2	ROCOv2_2023_train_000003	computed tomography of the chest showing the r...
3	ROCOv2_2023_train_000004	lateral view of the sacrum showing the low con...
4	ROCOv2_2023_train_000005	thoracic ct scan showing perihilar pulmonary l...

In [7]:
```
# pretrained BERT tokenizer
tokenizer_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(tokenizer_name)

print("BERT vocabulary size:", len(tokenizer))
```

```
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token
s), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 6.11kB/s]

config.json: 100% 570/570 [00:00<00:00, 73.7kB/s]

vocab.txt: 232k/? [00:00<00:00, 17.8MB/s]

tokenizer.json: 466k/? [00:00<00:00, 911kB/s]

```
BERT vocabulary size: 30522
```

In [8]:
```
# tokenize training captions
encoded_training_captions = tokenizer(
    training_caption_data["cleaned_caption"].tolist(),
    add_special_tokens=False,
    padding=False,
    truncation=False,
)
```

```
# calculate caption lengths
training_token_lengths = [
    len(token_ids) + 1
    for token_ids in encoded_training_captions["input_ids"]
]

maximum_caption_length = int(np.ceil(np.percentile(training_token_lengths, 99)))
maximum_caption_length = max(2, min(maximum_caption_length, tokenizer.model_max_length))

print("Maximum caption length:", maximum_caption_length)
```

```
Token indices sequence length is longer than the specified maximum sequence length for this model (610 > 512). Runnin
g this sequence through the model will result in indexing errors
Maximum caption length: 105
```

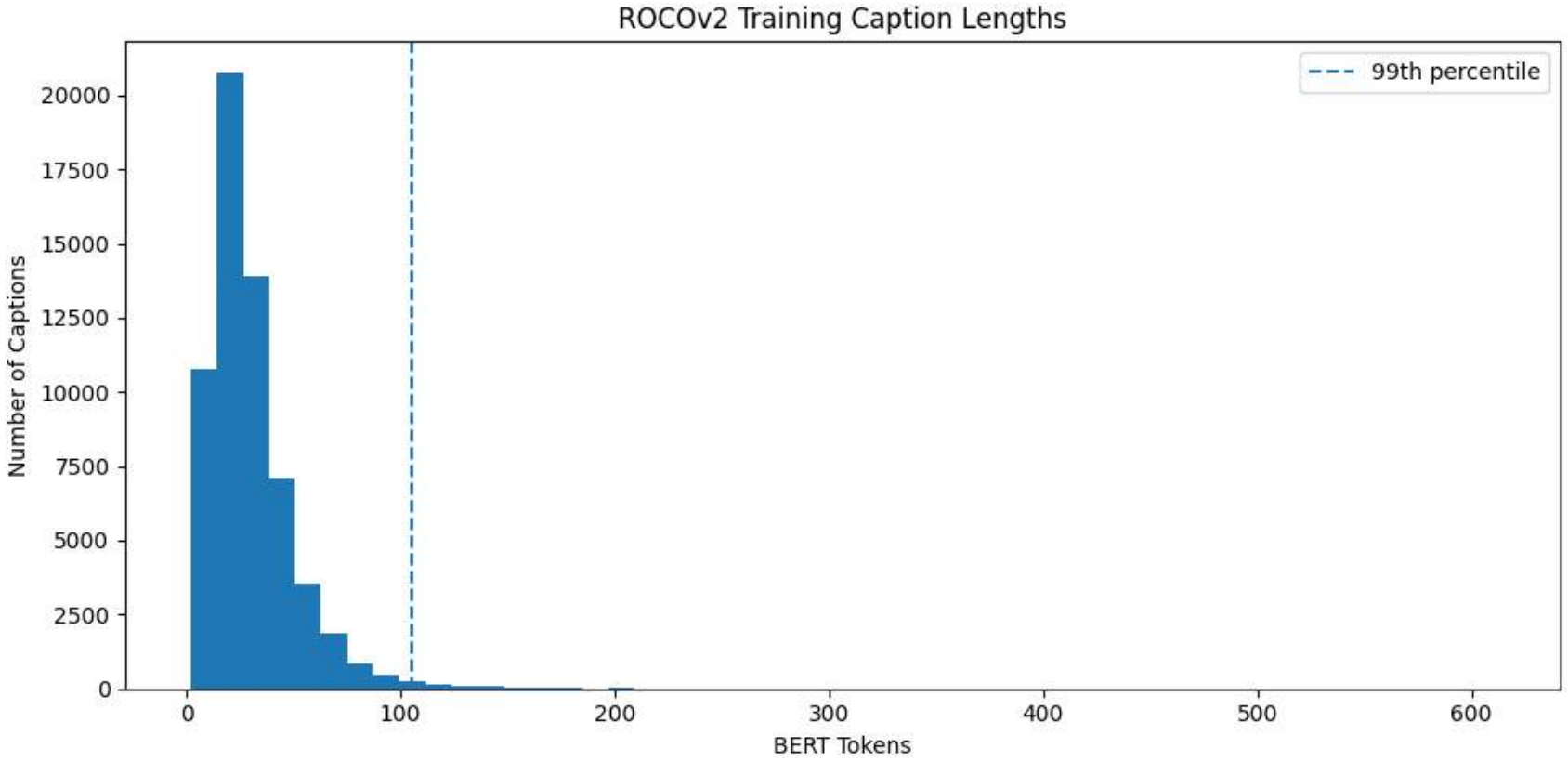
In [9]:

```
# display caption length statistics
display(pd.Series(training_token_lengths).describe())

plt.figure(figsize=(10, 5))
plt.hist(training_token_lengths, bins=50)
plt.axvline(maximum_caption_length, linestyle="--", label="99th percentile")
plt.title("ROCOv2 Training Caption Lengths")
plt.xlabel("BERT Tokens")
plt.ylabel("Number of Captions")
plt.legend()
plt.tight_layout()
plt.show()
```

	0
count	59958.000000
mean	30.365806
std	20.842734
min	2.000000
25%	17.000000
50%	26.000000
75%	38.000000
max	611.000000

dtype: float64


ROCOv2 Training Caption Lengths
99th percentile
Number of Captions
BERT Tokens
0
2500
5000
7500
10000
12500
15000
17500
20000
0
100
200
300
400
500
600

In [10]:
```python
# save BERT tokenizer and configuration
tokenizer_folder.mkdir(parents=True, exist_ok=True)
tokenizer.save_pretrained(tokenizer_folder)

tokenizer_configuration = {
    "tokenizer_name": tokenizer_name,
    "maximum_caption_length": maximum_caption_length,
    "decoder_start_token_id": tokenizer.cls_token_id,
    "end_token_id": tokenizer.sep_token_id,
    "pad_token_id": tokenizer.pad_token_id,
}

with open(tokenizer_folder / "transformer_configuration.json", "w", encoding="utf-8") as file:
    json.dump(tokenizer_configuration, file, indent=4)

print("Tokenizer saved:", tokenizer_folder)
```

```
Tokenizer saved: /content/drive/MyDrive/OCOM5300M_VLM_Benchmark/transformer/bert_tokenizer
```


Hugging Face (2026) *Transformers documentation*. Available At https://huggingface.co/docs/transformers/

Hugging Face (2026) *BERT model documentation*. Available At https://huggingface.co/docs/transformers/model_doc/bert